

An implementation of Open-RMF in an industrial environment

Michele Belletti

2025
December

Contents

I	Introduction	3
1	Background and motivation	5
2	Research questions and objectives	6
3	Scope and limitations	7
II	Literature Review	8
4	Overview of ROS1 and ROS2	10
4.1	Structure of ROS	10
4.2	Tools of ROS	12
5	Introduction to NAV2 and Google Cartographer	15
5.1	Description of Nav2	15
5.1.1	Cartographer	17
6	Review of related work on porting navigation stack from ROS 1 to ROS 2	19
7	Overview of open source fleet management systems, particularly Open-RMF	20
7.1	Open-RMF description	21

Part I

Introduction

This work presents a study on the new system, OPEN-RMF, which is still under active development but has reached a mature enough state to be deployed with robots at the Elettra Sincrotrone Trieste facility.

Chapter 1

Background and motivation

Elettra Sincontrone Trieste¹ is a major research facility that primarily provides users access to two light sources: the synchrotron Elettra and the free electron laser Fermi. In addition to operating these sources, the facility actively researches new technologies and techniques to enhance its capabilities and operations. Many recent advancements are based on robotics, which, especially after the COVID-19 pandemic and with the rapid rise of large language models (LLMs), have enabled new ways for users to interact with the infrastructure, including robot-assisted information retrieval and package delivery services. Previously, the facility developed a global planner, D-Star (D-star based optimized trajectory planner)[12], within the ROS 1 framework to enable robot navigation across large-scale environments. However, as ROS 1 has reached end-of-life, the codebase must now be migrated to ROS 2. Additionally, the facility employs robots from multiple vendors, each specialized in different tasks and not designed to coordinate with one another. In an environment that shares resources such as narrow passages and lifts, this lack of interoperability creates significant coordination challenges. To address this issue, Open-RMF² has been developed in recent years as a multiplatform system for controlling and managing heterogeneous fleets of robots.

¹<https://www.elettra.eu/>

²<https://www.open-rmf.org/>

Chapter 2

Research questions and objectives

The objectives of this thesis are as follows: to integrate a Rover Mini robot from the company RoverRobotics ¹ into the ROS 2 Nav2 ecosystem, to port the D-Star global planner from ROS 1 to ROS 2,



Figure 2.1: Rover Mini of the RoverRobotics

incorporate the robot into the Open-RMF framework for centralized fleet management, compare its performance to the ROS 1-based navigation stack, and evaluate the suitability of the Open-RMF ecosystem for deployment at the Elettra Sincrotrone Trieste facility.

¹<https://roverrobotics.com/>

Chapter 3

Scope and limitations

The principal limitation is the current lack of access to lift control systems, which prevents direct integration and operation by the robot fleet. Additionally, there is no automated system for opening and closing doors. While Open-RMF offers a high degree of customizability, and human-in-the-loop interactions can be configured—such as triggering a light or sending a notification when a robot approaches a door—this method requires a human to manually open the door or operate the lift. However, such an approach is not ideal for achieving full autonomy.

Part II

Literature Review

In this section, we provide a description of the key technologies used in this thesis. We begin by describing the Robot Operating System (ROS) and its functionality, followed by an overview of the navigation system employed, and a discussion of how the Open-RMF fleet management system operates.

Chapter 4

Overview of ROS1 and ROS2

The Robotics Operating System (ROS)¹ is an open-source collection of tools and libraries designed to support the development of robotics applications. Despite its name, ROS is not an operating system (OS) in the traditional sense, but rather a software development kit (SDK) that provides developers with a comprehensive set of tools for building robotic systems. It offers several features typically associated with an OS, including hardware abstraction, low-level device control, implementation of commonly used functionalities, inter-process communication, and package management. The ROS ecosystem is built upon the following core components:

- **Middleware communication layer (RMW):** Built on the Data Distribution Service (DDS) standard, this layer enables nodes to exchange information using an anonymous publish/subscribe pattern. This design promotes fault isolation, separation of concerns, and modular interfaces. ROS also supports synchronous, Remote Procedure Call (RPC)-style communication between services. While the default DDS implementation is eProsima Fast DDS (`rmw_fastrtps_cpp`), this work uses Eclipse Cyclone DDS (`rmw_cyclonedds_cpp`) due to its improved compatibility with the Nav2 navigation stack.
- **Development tools:** ROS provides a suite of tools for managing and enhancing software development, including utilities for launching, introspection, debugging, visualization, plotting, logging, and playback.
- **Multi-machine support** ROS includes tools and libraries for retrieving, building, writing, and executing code across multiple computers and OS.
- **Community and governance:** ROS is supported by an active and extensive global community, coordinated primarily by Open Robotics², which serves as the central organization for its development and maintenance.

4.1 Structure of ROS

The official ROS documentation³ provides comprehensive information about the system. All communication in ROS is handled through the **ROS Graph**, a peer-to-peer network of processes (called nodes) that execute concurrently and exchange

¹<https://www.ros.org/>

²<https://www.openrobotics.org/>

³<https://docs.ros.org/en/rolling/index.html>

information with one another. Some components of this graph are described below. Note that some of the functionalities described are specific to ROS 2 and are not available in ROS 1:

- **Nodes:** A node is a modular process that performs a specific task. Nodes communicate with each other through topics, services, actions, and parameters. In ROS 2, nodes can be implemented as **LifecycleNode**, which provide a managed state machine with defined transitions for startup and shutdown, enabling more deterministic behavior. Nodes can also be composed within the same process using the

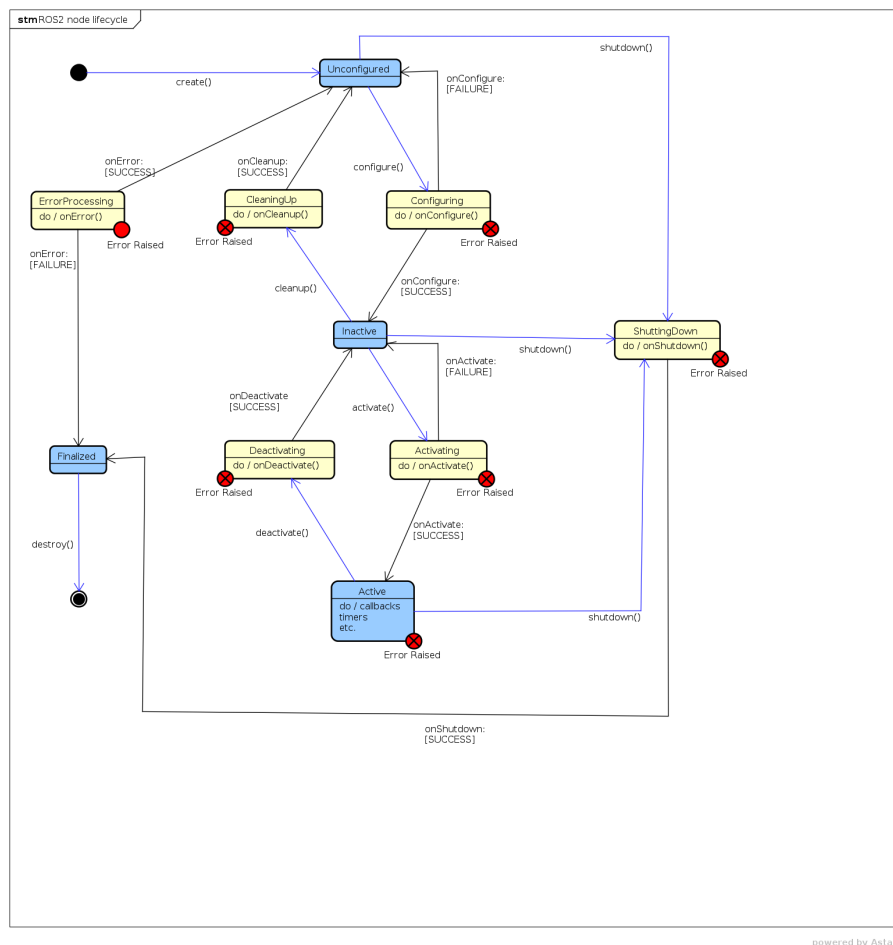
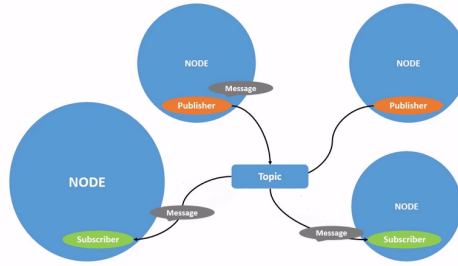


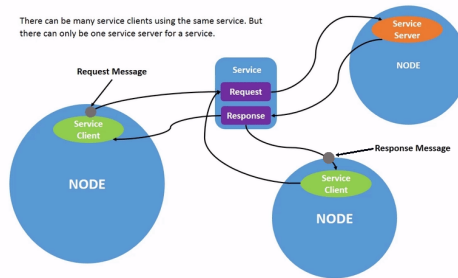
Figure 4.1: State Machine of LifecycleNode

Composition feature, which improves performance by bypassing middleware overhead[4].

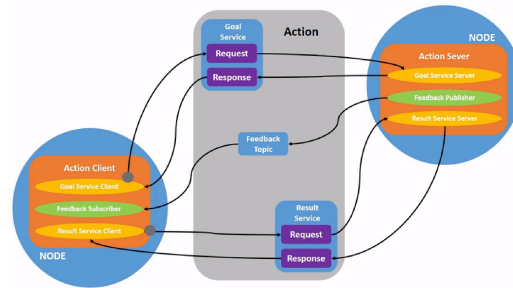
- **Topics:** Topics are used for communication between nodes to share streams of messages in a publisher-subscriber structure.

**Figure 4.2:** Topic process

- **Service:** Services are based on the request-response model. When a node with a service client sends a Request Message to a node with a service server, it receives a Response Message from that specific server.

**Figure 4.3:** Service process

- **Parameters:** Parameters are configurations of a node that can be modified without changing the source code.
- **Actions:** Actions are designed to support long-running tasks that require feedback and the ability to preempt. They combine two services (goal and result) and a publisher-subscriber mechanism (feedback). A node sends a goal request and receives an acknowledgment, then starts publishing feedback messages while executing the task, and finally responds with the result.

**Figure 4.4:** Action process

4.2 Tools of ROS

Now, I will describe the tools that are built into ROS for developing robotic systems:

- **Launch:** A file structure that allows the simultaneous start of multiple nodes with a set of parameters.

- **rqt**: A graphical user interface (GUI) tool for ROS. It includes a series of plugins that facilitate development, monitoring, and diagnostics of the ROS system.
- **rqt_console** A plugin for rqt that visualizes the logs generated by the ROS.
- **Rviz**: A 3D visualization tool for the Robot Operating System. It allows visualization of sensor data and state information from ROS.

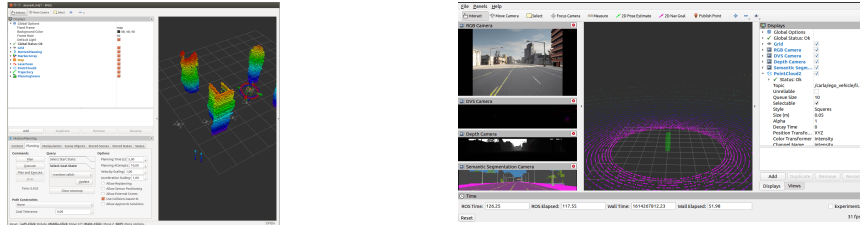


Figure 4.6: Examples of visualization of ROS data with Rviz

- **roscdep**: A command-line utility for installing system dependencies required by ROS packages.
- **sros2**: A security extension for ROS 2 that provides tools and instructions to apply security policies to DDS (Data Distribution Service).

Another important category of tools that are integrated with ROS is simulation environments. These environments allow developers to test their code on simulated robots under various conditions, which helps identify bugs that could potentially harm a real robot or its surroundings. Simulation tools provide realistic physical behavior for all components of a robotic system. According to the ROS documentation, two main simulation tools are commonly used: Gazebo and Webots. Among them, Gazebo is also maintained by Open Robotics and is widely adopted in many projects, including the one discussed in this paper. Gazebo offer:

- **ROS Integration**: A bridge that converts Protobuf messages to ROS messages, enabling communication between Gazebo and ROS.
- **Performance tools**: Features for distributed simulation across servers, dynamic asset loading, and adjustable time steps to optimize simulation performance.
- **Realistic simulation**: Support for a variety of sensors, including their noise models, within a 3D world rendered using OGRE 2.1⁴ and powered by the DART5⁵ physics engine.
- **Extensibility**: A modular design that supports plugins, allowing developers to run custom code that interacts directly with the simulation environment.

⁴<https://wiki.ogre3d.org/Home>

⁵<https://dartsim.github.io/>

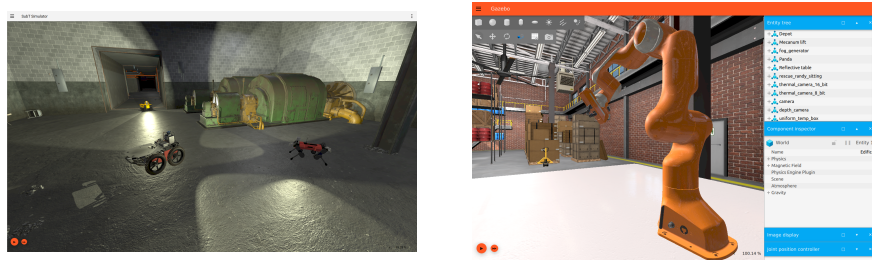


Figure 4.8: Examples of Gazebo

Chapter 5

Introduction to NAV2 and Google Cartographer

Multiple tools are available to enable a robot to move autonomously within an environment. One of the most complete systems is Nav2 ¹ [7], the successor to the ROS 1 Navigation Stack.

5.1 Description of Nav2

Nav2 provides a comprehensive set of tools for perception, planning[9], control, localization, and visualization, allowing any robot to navigate autonomously and reliably, even in complex environments. It enables building a model of the environment from sensor data, dynamically calculating the optimal path to avoid obstacles, setting motor velocities, and executing navigation tasks using Behavior Trees.

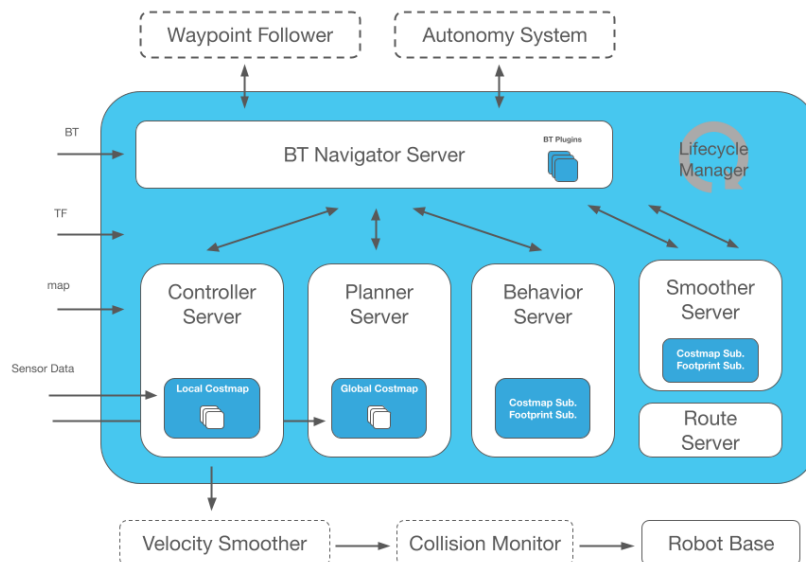


Figure 5.1: Nav2 architecture

Nav2 introduces several improvements over ROS 1, including:

¹<https://nav2.org/>

- **Behavior trees**[1]: Nav2 uses Behavior Trees to structure and execute tasks in a human-readable and modular way. These trees enable the creation of complex systems and can be tested with various tools. The BehaviorTree.CPP² library is used to implement them, and it allows linking different trees for flexible behavior management.
- **nav2_util LifecycleNode**: This is a wrapper around the standard `rclcpp_lifecycle::Lifecycle` described earlier 4.1. It adds a bond connection to the Lifecycle Manager, which monitors the state of each node and can safely transition nodes through their lifecycle states in the event of a failure.
- **Action Server**: As described in 4.1, Nav2 uses action servers for Behavior Tree interactions, such as `NavigationToPose` (navigate to a specific location) and `NavigationThroughPoses` (navigate through multiple waypoints).

Nav2 is composed of several key nodes that work together to control the robot:

- **Controller Server**: Handles movement requests. It uses plugin-based modules such as a progress checker (to verify if the robot is making progress), goal checker (to determine if the robot has reached its destination), and a controller (to generate velocity commands using the **local costmap** while avoiding collisions).
- **Planner Server**: Computes the global path to the destination using sensor data and a plugin-based global planner loaded with the **global costmap**.
- **BT Navigator Server**: Implements the `NavigateToPose`, `NavigateThroughPoses`, and other task interfaces. It is a Behavior Tree-based implementation of navigation that is intended to allow for flexibility in the navigation task and provide a way to easily specify complex robot behaviors.

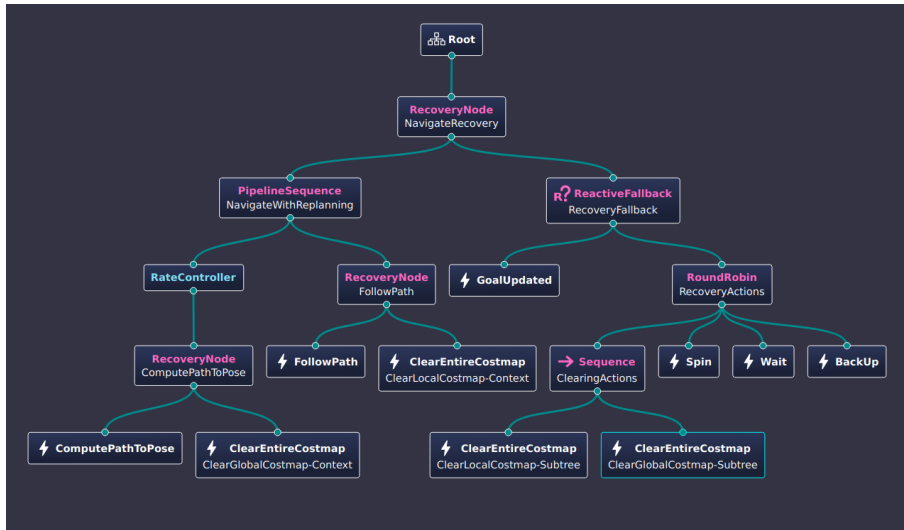


Figure 5.2: Humble Nav2 BT navigation `navigate_to_pose_w_replanning_and_recovery.xml` loaded with Groot2

²<https://www.behaviortree.dev/>

- **Smooth Server:** Smooth the path minimizing sharp turns and rapid rotations.
- **Behavior Server:** Handles specific behavior actions like spinning, backing up, and waiting.
- **Waypoint Follower:** An example of orchestrated control, this node takes a list of waypoints and uses the NavigateToPose action server to move between them, optionally performing custom tasks (e.g., taking a photo) at each waypoint through plugins. It can be replaced by any **Autonomy System**
- **Route Server:** Calculates routes through a predefined navigation graph instead of using free-space planning.
- **Velocity Smoother:** Smooths velocity, acceleration, and deadband signals sent to the robot to ensure safe and smooth motion.
- **Costmap** is a representation of the environment using a regular 2D grid of cells with a cost (unknown, free, occupied, or inflated) and is used for generate the global plan or to compute local control efforts.

The next things that Nav2 relies on for controlling the navigation are:

- **state estimation** of the robot that generate a suitable TF tree, a tree structure of coordinate frames generated using the transform library[2], based on the REP 105, that is should provide this structure: `map` $\rightarrow$ `odom` $\rightarrow$ `base_link`. According to the REP 105 they are defines as:
 - `map` : A world-fixed frame that may change in discrete jumps. It's useful for long-term global reference but not suitable for short-term localization.
 - `odom` : A continuous world-fixed frame that may drift over time. It provides short-term local reference.
 - `base_link`: A robot fix frame.

The `odom` $\rightarrow$ `base_link` link can be estimated using various sensors. For improved accuracy, we use the `robot_localization` package, which provides nonlinear state estimation using Kalman Filter [8].

The `map` $\rightarrow$ `odom` link can be generate with a global positioning system like Simultaneous Localization And Mapping (SLAM) method or using the one provided by Nav2, Adaptive Monte-Carlo Localization (AMCL).

- **map** can either be preloaded or built dynamically using a SLAM method. One SLAM method systems is Google Cartographer, which will be described in the next section. It is also the SLAM method that was used to generate the map for navigation.

5.1.1 Cartographer

Cartographer[3] is a SLAM method that operates using two main subsystems: a **Local SLAM** (called **Frontend**) that build a series of locally consistent **submaps** from successive scans and the pose extrapolator but with the possibility that it can

drift during acquisition and a **Global SLAM** (called **Backend**) reduces drift by performing loop closure—detecting when the robot has returned to a previously visited location. It uses scan matching against submaps and applies global optimization to align all submaps consistently and build a globally accurate map. Cartographer is capable of achieving mapping accuracy within 2–3 cm, but it requires extensive parameter tuning to reach optimal performance.

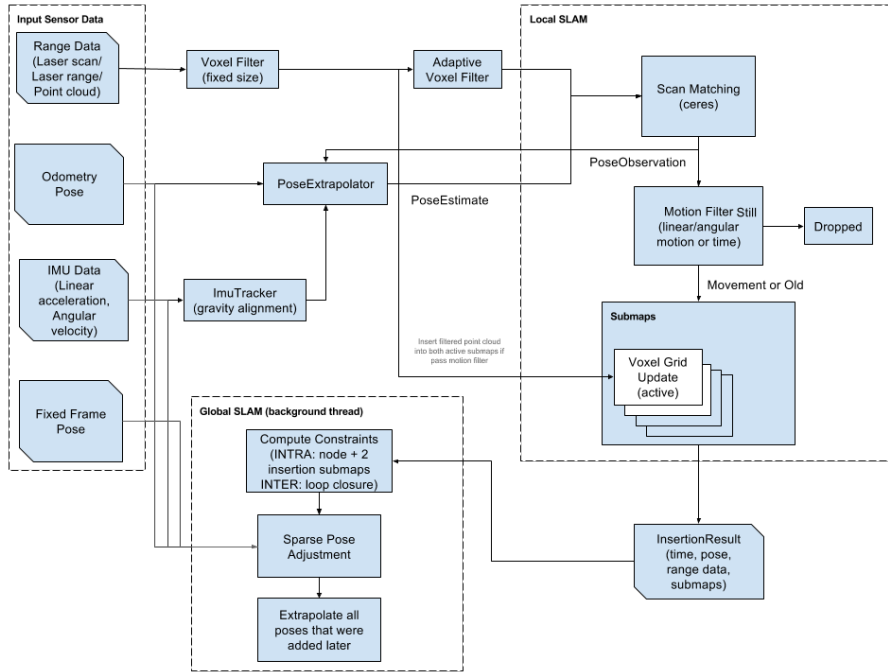


Figure 5.3: Cartographer Algorithm

Chapter 6

Review of related work on porting navigation stack from ROS 1 to ROS 2

The big diffusion of ROS as a software for controlling robots bring new users cases that were not considered when was built. Starting as an academic project, the new cases that has met are the need of a way to control a fleet of them, deveoping on little embedded board and microcontrollers, real time system, not ideal network and the code was not designed for industrial usage. These changes made impossible to save ROS from a total rewrite of the code that brigh to have ROS 2, all more documentation is accessible at <https://design.ros2.org/> . Multiple company started using the ROS 2 ecosystem and how it increase their development[6] and also in the Nav2[9]. New projects are born like micro-ROS ¹ for controlling microcontrollers. And the new tool Node Composition allow an higher degree of performance optimization [4] and also allow for a better security[11] While more detailed information about how to change the code are described in these sources, like the official documentation² and also secondary ³ and are the tools that are going to be used in the porting of the global planner. **The notable changes, also described in that paper are:**

Like describe in this paper the use of ROS 2 respect ROS 1 bring many advantages and is the way to go.

¹<https://micro.ros.org/>

²<https://docs.ros.org/en/rolling/How-To-Guides/Migrating-from-ROS1.html>

³https://industrial-training-master.readthedocs.io/en/melodic/_source/session7/ROS1-to-ROS2-porting.html

Chapter 7

Overview of open source fleet management systems, particularly Open-RMF

Currently these are some the fleet management system available:

- **Isaac Mission Dispatch**¹ is a NVIDIA microservice enable fleet management software to submit missions to multiple robots and monitor the robot and mission status. It provides a connection between the robots and the fleet management system but does not handle logistics such as task allocation or conflict resolution, e.g., robots with intersecting paths. The implementation relies VDA5050 protocol as an industry standard between a cloud control service and mobile robots, and uses MQTT as a lightweight, publish-subscribe, machine to machine network protocol designed for devices with resource constraints and limited network bandwidth.
- **ROOSTER Fleet Management**² is a ROS 1 based open source project to develop a heterogeneous fleet management solution with task allocation, scheduling and routing capabilities.
- **Open Robotics Middleware Framework (Open-RMF)**³ is a free, open source, modular software system that enables sharing and interoperability between multiple fleets of robots and physical infrastructure, like doors, elevators and building management systems.
- **Robofleet**⁴[10] based on ROS 1 is open-source system which provides inter-robot communication, remote monitoring, and remote tasking for a heterogeneous fleet of ROS-enabled service-mobile robots that is designed with the practical goals of resilience to network variance and security control in mind.

While others gives an infrastructure for start the control of a fleet:

- **Transitive Robotics**⁵ is an open-source framework for full-stack robotics. It is designed to make it easy to create capabilities that seamlessly connect

¹https://github.com/NVIDIA-ISAAC/isaac_mission_dispatch

²<https://github.com/ROOSTER-fleet-management>

³<https://www.open-rmf.org/>

⁴<https://github.com/ut-amrl/robofleet>

⁵<https://transitiverobotics.com/>

robot, cloud, and web front-ends including a dynamically updating shared data model. It offers solutions to many problems developers encounter when building full-stack robotic applications, including unreliable networks, robots that are regularly offline or turned off, as well as cross-device version.

- **Robot Web Tools**⁶ is a collection of tools that allow to communicate with a remote robot.

Given that Open-RMF gives a control over multiple fleet, a structure for them to interact with each other to obtain the best outcome, a lot of tool to build the system, is based on ROS 2 and is open source, it is going to be the base on witch we will build our fleet. Now we will all the capabilities of Open-RMF

7.1 Open-RMF description

Open-RMF is a flexible tool set build on ROS 2 for allowing the interaction between heterogeneous robot fleets. It uses a standardized protocol for communication with infrastructures, environments and automations to optimize the use of the critical resources (like other robots, lifts, doors, narrow passages) of a system to control the traffic. It gives intelligence to a system avoiding conflicts for the available resources and assigned it correctly. This figure describe how works Open-RMF:

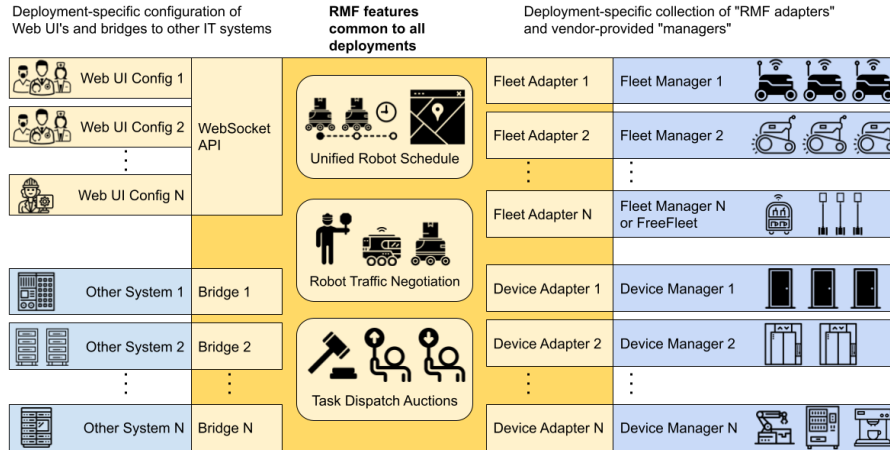


Figure 7.1: Open-RMF Structure

There are 3 type of fleet adapter :

- *Full Control*: have full status updates and full control on the path
- *Traffic Light*: have full status updates and a pause/resume control
- *Read Only*: have only status updates. It can be only one of this type because more of them will make avoiding deadlock or resource conflict nearly impossible.

In case there is a fleet with *No Interface* this can result deadlocks on the available resources and is not compatible with a RMF system.

The Open-RMF core `rmf_core` is built on this list of packages:

⁶<https://robotwebtools.github.io/>

- **rmf_traffic**: Core scheduling and traffic management systems based on the following traffic deconfliction: *Prevention* and *Negotiation* using a living platform-agnostic Traffic Schedule Database.

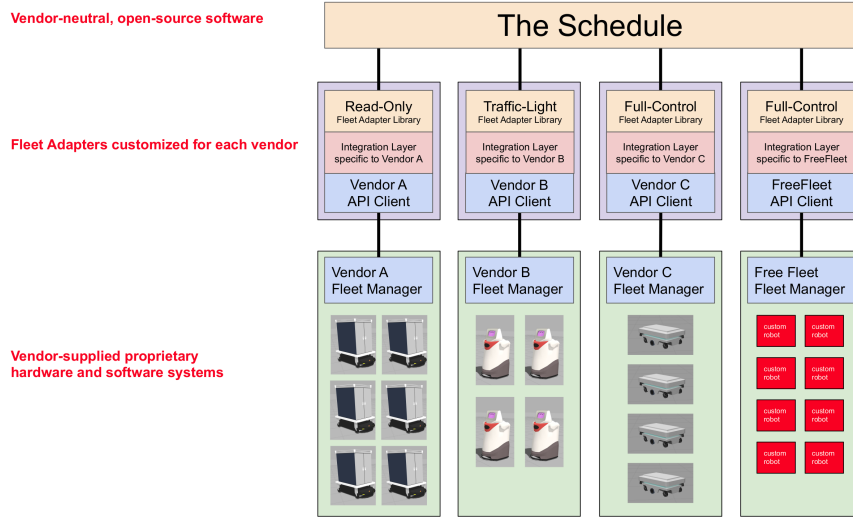


Figure 7.2: Open-RMF Schedule structure

- *Prevention* when there are some changes in the DB, like cancellations, the fleet manager can plan the routes to avoid conflicts.
- *Negotiation* When thought the Database there is an upcoming conflict between two or more schedule participants, it will send a conflict notice out to the relevant fleet managers, that will reply with their preferable path that can accommodate the others and through a external judge choose the best path to follow

Conflict avoidance for automated guided vehicle AGV's is implemented with time-dependent extension to A* search for both path and task. This search takes time into account so that it can find paths through space and time that account for the motions of other agents that are in the traffic schedule. It's not designed for autonomous mobile robot (AMR) that is in development.

- **rmf_traffic_ros2**: provide the interface between rmf_traffic and ros2
- **rmf_task**: APIs and base classes for defining and managing Tasks in RMF. RMF supports three types of task requests out of the box:
 - *Clean*: For robots capable of cleaning floor spaces in facilities
 - *Delivery*: For robots capable of delivering items between locations in facilities
 - *Loop*: For robots capable to navigating back and forth between locations in facilities
- **rmf_dispatcher_node**: RMF simplifies task allocation and management across multi-fleet systems. When a user submits a new task request, RMF will intelligently assign it to the robot in the fleet that can best perform the task as described in the image.

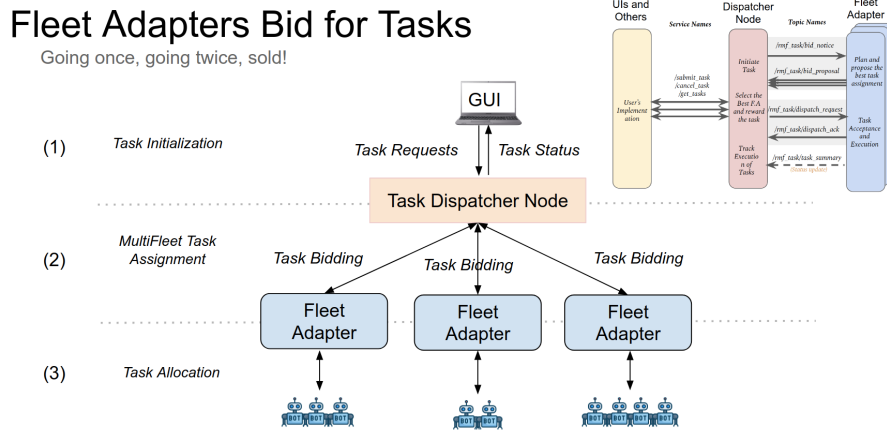


Figure 7.3: Open-RMF Bid Task

- **rmf_battery:** rmf battery estimation
- **rmf_ros2:** ros2 adapters and nodes and python bindings for rmf_core
- **rmf_utils:** utility for rmf

For the development these are the tools ready to use to develop:

- **RMF demos** contains demonstrations of the capabilities and are a starting point for the development.
- **Traffic Editor** is a graphical editor that allow to create a map with traffic patterns and the place of interaction that RMF will use to control the traffic. It also can generate a Gazebo world for simulation.

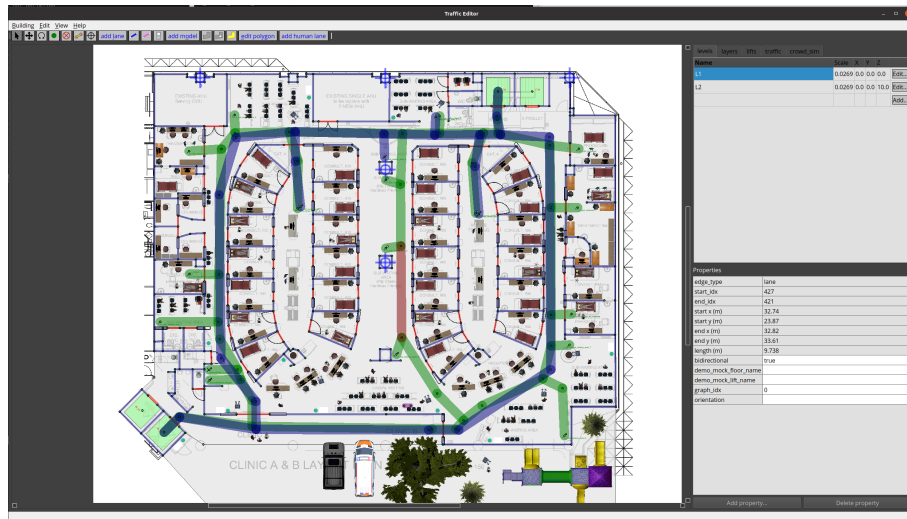


Figure 7.4: Traffic Editor

- **Free Fleet** is a free fleet adapter allow to integrate standalone mobile robots into Open-RMF to form a heterogeneous fleet.
- **RMF Schedule Visualizer** is a plugin for rviz to see and control Open-RMF.

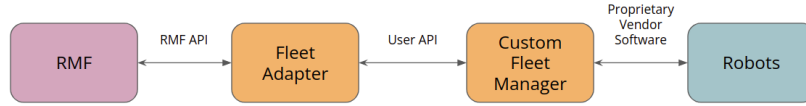


Figure 7.5: Fleet Adapter

- **RMF Web UI** is a web application for visualization and control the Robotic Middleware that is `rmf_core`

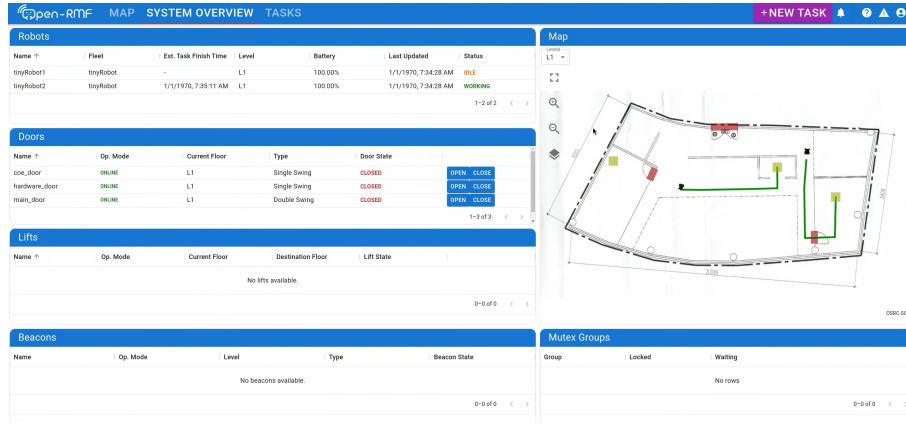


Figure 7.6: RMF Web

- **RMF Simulation** plugins to simulate each capability in gazebo.
- **Simulation assets** are the models freely available.

today the tool for converting the data are <https://www.ros.org/blog/noetic-eol/>
implement also planning benchmark

describe and **use** composition in nav2 to obtain better performance [5]

Inoltre per gestire il traffico sulla rete utilizzeremo zenoh (Zero Overhead Network Protocol) ed il bridge per ros2 zenoh-bridge-ros2dds che è pub/sub/query protocol unifying data in motion, data at rest and computations. It elegantly blends traditional pub/sub with geo distributed storage, queries and computations, while retaining a level of time and space efficiency that is well beyond any of the mainstream stacks. data liberator protocol. Zenoh liberates data in several dimensions.

with chatgpt possiamo gestire il robot in due modi: si ferma ed esegue l'azione non comunicando con rmf (la linea rimane bloccata dal suo passaggio)

Bibliography

- [1] Michele Colledanchise and Petter Ögren. *Behavior Trees in Robotics and AI*. July 2018. DOI: 10.1201/9780429489105. URL: <http://dx.doi.org/10.1201/9780429489105>.
- [2] Tully Foote. “tf: The transform library”. In: *Technologies for Practical Robot Applications (TePRA), 2013 IEEE International Conference on*. Open-Source Software workshop. 2013, pp. 1–6. DOI: 10.1109/TePRA.2013.6556373.
- [3] Wolfgang Hess et al. “Real-Time Loop Closure in 2D LIDAR SLAM”. In: *2016 IEEE International Conference on Robotics and Automation (ICRA)*. 2016, pp. 1271–1278.
- [4] Steve Macenski et al. *Impact of ROS 2 Node Composition in Robotic Systems*. 2023. arXiv: 2305.09933 [cs.R0]. URL: <https://arxiv.org/abs/2305.09933>.
- [5] Steve Macenski et al. “Impact of ROS 2 Node Composition in Robotic Systems”. In: *IEEE Robotics and Automation Letters* 8.7 (2023), pp. 3996–4003. DOI: 10.1109/LRA.2023.3279614.
- [6] Steven Macenski et al. “Robot Operating System 2: Design, architecture, and uses in the wild”. In: *Science Robotics* 7.66 (2022), eabm6074. DOI: 10.1126/scirobotics.abm6074. URL: <https://www.science.org/doi/abs/10.1126/scirobotics.abm6074>.
- [7] Steven Macenski et al. “The Marathon 2: A Navigation System”. In: *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2020.
- [8] T. Moore and D. Stouch. “A Generalized Extended Kalman Filter Implementation for the Robot Operating System”. In: *Proceedings of the 13th International Conference on Intelligent Autonomous Systems (IAS-13)*. Springer, 2014.
- [9] DV Lu A. Merzlyakov M. Ferguson S. Macenski T. Moore. “From the desks of ROS maintainers: A survey of modern & capable mobile robotics algorithms in the robot operating system 2”. In: *Robotics and Autonomous Systems* (2023).
- [10] Kavan Singh Sikand et al. *Robofleet: Open Source Communication and Management for Fleets of Autonomous Robots*. 2021. arXiv: 2103.06993 [cs.R0]. URL: <https://arxiv.org/abs/2103.06993>.
- [11] Victor Mayoral Vilches et al. *SROS2: Usable Cyber Security Tools for ROS 2*. 2022. arXiv: 2208.02615 [cs.CR]. URL: <https://arxiv.org/abs/2208.02615>.

-
- [12] Andrey Vukolov et al. “Universal Configurable Navigation and Control System for Industrial Unmanned Ground Vehicles with Differential Chassis”. In: *Advances in Mechanism and Machine Science*. Ed. by Masafumi Okada. Cham: Springer Nature Switzerland, 2023, pp. 287–301. ISBN: 978-3-031-45770-8.